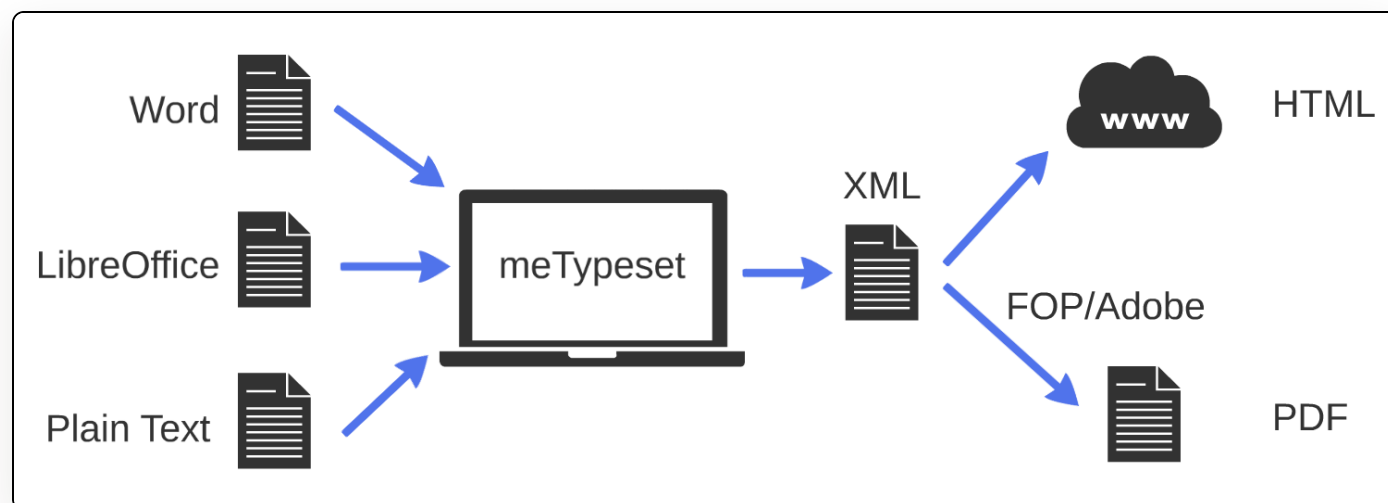


Building a real XML-first (XML-in) workflow for scholarly typesetting

Cite as: Martin Paul Eve, "Building a real XML-first (XML-in) workflow for scholarly typesetting", *eve.gd: Martin Paul Eve*, July 20, 2015, <https://doi.org/10.59348/w78n5-he827>.



For some time now, I’ve worked to build an open-source JATS XML typesetter. It’s called meTypeset. It’s not by any means perfect and the approach it takes is unlikely to ever yield 100% good markup from Word input. It does, though, get it right a lot of the time in “basic” (in terms of underlying XML semantic complexity) rich-text documents, which is what most academics (especially in the humanities) are writing. It also plugs into Zotero and will produce nice element-citation blocks for you.

Part of the work on the Mellon grant that Birkbeck has been awarded consists of making a user-friendly interface to meTypeset and a self-diagnostic mode. This will include a “triage” mode; the software will tell you whether or not it is capable of parsing the input document. This should result in a workflow where, at least if meTypeset can’t handle it in-house, we can pay for the labour of doing the typesetting the old, hard way (eXtyles etc.).

In my work on [Orbit](#), I also developed a second component to meTypeset, called [meXml](#). This is a set of XSLT documents that will transform the underlying JATS into human-readable forms, most notably HTML and PDF. HTML is relatively easy here. We take the input document and spit out another plain-text document with slightly re-worked tags, a header, reference list formatting etc. No sweat. PDF is much harder though.

Aside from all [the traditional complaints about the PDF](#), it's pretty hard to create a PDF from an XML document. But researchers continue to want them. Indeed, it turns out, to my horror, that most professional typesetters don't actually work in an XML-first flow. Instead, once the Word document has been marked-up (via a plugin), this is passed to different processes (such as Adobe InDesign) and the XML is spat out at the end. This is called an XML-out workflow. By contrast the mode in which I have been interested is called XML-in (or XML-first) and, borrowing a diagram from [Jonathan McGlone's excellent article on the topic](#) (CC BY-SA license), it looks a little like this:

XML-Out



XML-In



To-date, the way in which I have achieved an XML-first workflow for Orbit has been to run the JATS XML document through a two-stage process. Firstly, I have developed extensively an XSLT document that creates an XSL-FO intermediary form. XSL-FO stands for Extensible Stylesheet Language Formatting Objects and the resulting XSL-FO document can be passed to a Formatting Objects Processor. The implementation that I have used to do this is called Apache FOP. The current workflow looks like this:

Here's the problem, though. XSL-FO was discontinued: the last update for the Working Draft was in January 2012, and its Working Group closed in November 2013. A recent conference held in Prague on XML typesetting technologies (Saturday February 14th 2015; I wasn't there, I saw via Twitter) also noted that:

- XSL-FO is “Too complicated, too verbose”
- There is a “lack of experienced XSL-FO developer”
- “Publishers are on the move away from XSL-FO”
- “XSL-FO is going to die (slowly)”

There are also specific problems with Apache's implementation of Formatting Objects rendering (in FOP). Apache FOP is capable of handling basic layout formats but is not adept at producing more complex layouts; tables are particularly badly handled. Font support in Apache FOP is poor and the software is unable to substitute glyphs at the character level when the primary font does not support a necessary character (for instance, accents in non-English language texts).

Furthermore, designing XSL-FO layouts is technically difficult and time-consuming.

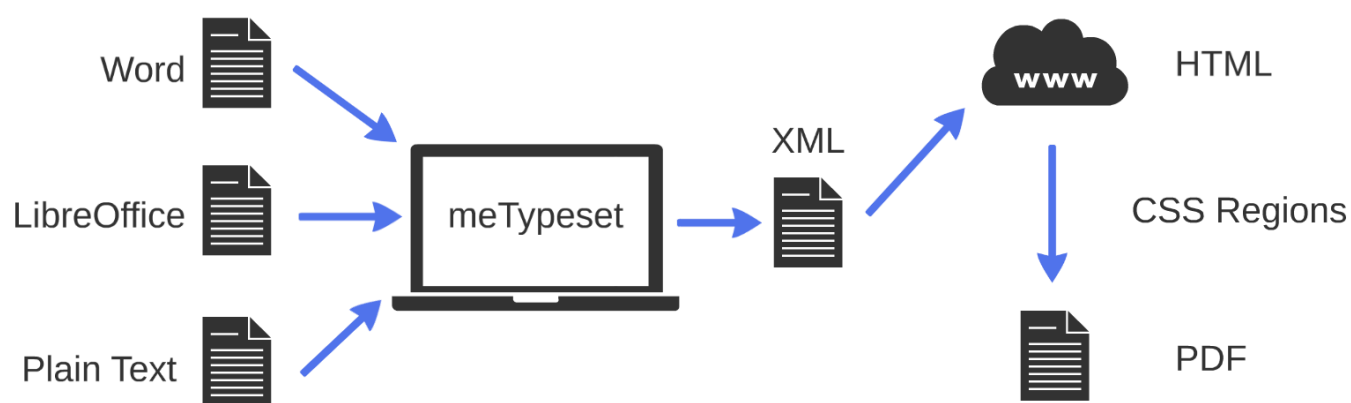
CSS Regions as an alternative approach

So what might be a viable alternative flow using open source software that we could develop? There are certainly alternative technologies that are well-supported, that are not overly complicated, for which there are broad, existing developer bases and that will mitigate the slow death of XSL-FO. That technology is, namely, Cascading Style Sheets (CSS).

While we traditionally think of CSS as pertaining to online media, we also know that all major browsers have the ability to print to PDF. There are also a range of command-line tools that can render a webpage in the background and save the PDF to disk when done. If we could render an HTML document, using advanced CSS, to *look exactly like a PDF* with the *precise dimensions of the page*, then we would have solved the problem because we could easily create a PDF automatically from an XSLT transform to HTML in an XML-in/XML-first workflow.

How, though, can we control an HTML document so that it looks exactly like a PDF? This is easier said than done, but there is a pagination technology for HTML called “CSS Regions”. Originally proposed by Adobe, Regions is not well loved as a standard. In fact, virtually no browser supports it. But, we *can* get Regions support by [embedding a piece of Javascript in a page](#). I don’t claim to be the first to have come with this as an idea; [Adam Hyde put me on to it](#) a long time ago via one of his excellent blog posts. CSS Regions [are considered harmful](#) by some commentators, but that is for general web consumption. If we’re using them in a back-end production environment, all those aspects of harm disappear, especially when compared to Formatting Objects approaches. In fact, one of Adobe’s [original posts on the CSS Regions](#) says that they “expect CSS Regions to be used in on-screen paginated presentations”. They also released [a series of CSS Regions pagination templates](#) as a starting point, on which we can build.

CSS Regions is a technology that allows the designer to create fixed region spaces and to then flow arbitrary content between these regions (a so-called “region chain”). If one creates a set of sequential regions of 8.3x11.7 inches and flows the content between them, then one has created a series of pages that, when a browser prints to PDF, yields a paginated output. This is the technology that we have started to build and will shortly begin releasing snapshots of. The workflow now looks like this:



Progress so far has been pretty good and we have a working layout engine. In fact, here’s a [sample cover page for an article](#). This is much more elegant and standards-friendly than the FOP version we were using before. The current version we have takes about 0.7 seconds per page to re-flow

content. Most of this time is due to the non-native Javascript implementation, sadly, but until/if there's decent browser support, this polyfill approach will work just fine.

While, obviously, much more of the mechanics will be revealed when we start releasing code (which will be daily once it's clean enough to be public), I'll just share a few of the technicalities that I encountered getting this to work so far. The major aspect that I broached yesterday was the problem that we can't know, in advance, how many pages we'll need. This means we need to dynamically add page regions to the HTML document. The way I did this was to setup a document with 50 pages initially and then set a hook to the "regionfragmentchange" event, which is fired when the first flow layout is complete.

```
document.getNamedFlows()[0].addEventListener('regionfragmentchange', modifyFlow);
```

Then, when this event fires in the modifyFlow function, we can use document.getNamedFlows()[0].firstEmptyRegionIndex to ascertain if there are remaining empty pages or if they are all full. If document.getNamedFlows()[0].firstEmptyRegionIndex is -1 when that fires, then it needs more pages added, and I do so in increments of 10. If the value of that property isn't -1, then I remove empty pages down to the last filled region. It's far quicker to initially over-populate pages than to under-populate and then add, but this does stack up on the CPU time when you go > 50 pages, so I compromised there.

All content on this page licensed under a CC BY-SA license.